

Sprint 2 Structured Notes: Terminal, Shell, and Command Line Basics

1. Current Progress

You are now moving from **Sprint 1** to **Sprint 2**.

In **Sprint 1**, you learned about the basic tools needed before learning databases. You focused on:

- VS Code
- Workspace
- Project folder
- `index.html`
- Live Server
- Basic shortcuts
- Beginner extensions

Sprint 1 was about preparing your coding environment.

Now you are starting **Sprint 2**.

Sprint 2 is about understanding the **terminal**, **command line**, and **shell**.

The full learning path is:

Tools → Terminal → Bash → Bash Scripting → Git → PostgreSQL → SQL →
Normalization, Security, Performance

So far:

Sprint 1 = Tools
Sprint 2 = Terminal basics

You are not learning SQL yet. Before SQL, PostgreSQL, Git, and Bash scripting, you first need to feel comfortable typing commands.

2. Main Goal of Sprint 2

The main goal of Sprint 2 is:

Understand what you are using when you type commands.

By the end of Sprint 2, you should be able to explain these three things in simple words:

1. What is a **terminal**?
2. What is a **command line**?
3. What is a **shell**?

You should also be able to use basic terminal actions:

```
Up arrow  
Down arrow  
Tab  
Ctrl + L  
cls  
Ctrl + C  
Ctrl + Z  
fg  
!!
```

Sprint 2 is not about writing big commands yet. It is about understanding the environment where commands are typed and run.

3. Why Sprint 2 Is Important

Many beginners are afraid of the terminal because it looks technical.

But the terminal is only a text-based way to control the computer.

You will use the terminal later for:

- Creating folders
- Moving between folders
- Creating files
- Running Bash commands
- Running Bash scripts
- Using Git
- Connecting to PostgreSQL
- Running SQL tools
- Managing project files

So Sprint 2 gives you the foundation for many future sprints.

Simple meaning:

If you understand Sprint 2 well, Bash, Git, and PostgreSQL will become easier later.

4. What Is the Command Line?

The **command line** is the place where you type commands.

Instead of clicking buttons with the mouse, you type text commands and press **Enter**.

Example:

```
echo "Hello"
```

This command prints:

```
Hello
```

Simple meaning:

The command line is a text-based place where you tell the computer what to do.

Another simple explanation:

```
Clicking method = use mouse and buttons  
Command line method = type command and press Enter
```

Example:

Instead of opening a folder by clicking, you can type a command later such as:

```
cd my-project
```

You will learn file and folder commands properly in Sprint 3.

For Sprint 2, just understand that the command line is the input area where commands are typed.

5. What Is a Terminal?

A **terminal** is the application or window that gives you access to the command line.

It is the program you open when you want to type commands.

Simple meaning:

The terminal is the app or window where you type commands.

Think like this:

```
Terminal = the window  
Command line = the place inside the window where you type
```

Examples of terminals on Windows

Windows users may use:

```
Command Prompt  
PowerShell  
Windows Terminal  
Git Bash
```

Examples of terminals on macOS

macOS users may use:

```
Terminal  
iTerm
```

Examples of terminals on Linux

Linux users may use:

```
Terminal  
Kitty  
Ghostty
```

Different systems may have different terminal applications, but the basic idea is the same.

6. What Is a Terminal Emulator?

A **terminal emulator** is an application that gives you a terminal-like interface on a modern computer.

In old computers, a terminal was a physical machine used to communicate with another computer.

Today, we usually use a software window that behaves like a terminal. That software is called a terminal emulator.

Simple meaning:

A terminal emulator is a modern terminal app that lets you type commands.

Examples:

```
Windows Terminal  
iTerm  
Kitty  
Ghostty
```

Beginner note:

Most people simply say **terminal**, even when technically they are using a **terminal emulator**.

For this course, you do not need to worry too much about the difference. Just remember:

```
Terminal / terminal emulator = the app window used for commands
```

7. What Is a Shell?

A **shell** is the program that reads your command, understands it, and runs it.

When you type a command, the shell interprets it and gives the result back.

Example:

You type:

```
echo "Hello"
```

The shell reads the command and prints:

```
Hello
```

Simple meaning:

The shell is the command interpreter.

Another simple meaning:

The shell understands the command you typed and tells the computer what to do.

Common shells include:

```
Bash
PowerShell
Zsh
Command Prompt
```

Important point:

The terminal and shell are connected, but they are not exactly the same.

The terminal is the window.

The shell is the program inside that understands commands.

8. What Is Bash?

Bash means:

```
Bourne Again SHell
```

Bash is a common shell used in Unix-like environments.

You may find Bash in:

- Linux
- macOS
- Git Bash on Windows
- Many servers
- Developer environments

Simple meaning:

Bash is one type of shell.

You will learn Bash commands in Sprint 3 and Bash scripting in Sprint 4.

For now, remember:

```
Shell = general name  
Bash = one specific shell
```

Example comparison:

```
Vehicle = general name  
Car = one type of vehicle  
  
Shell = general name  
Bash = one type of shell
```

9. Terminal vs Command Line vs Shell

This is the most important concept in Sprint 2.

Term	Simple Meaning
Terminal	The app or window you open
Command line	The text area where you type commands
Shell	The program that understands and runs your commands

Simple restaurant example

Imagine a restaurant.

```
Terminal = restaurant building  
Command line = place where you give your order  
Shell = waiter who understands your order  
Computer / operating system = kitchen that does the real work
```

When you type a command:

```
You give the order → shell understands it → computer does the work → terminal  
shows the result
```

Simple computer example

```
Terminal = window  
Command line = typing area  
Shell = command interpreter
```

Good beginner answer:

```
A terminal is the app I open.  
The command line is where I type commands.  
The shell is the program that understands and runs those commands.
```

10. How a Command Works

When you type a command, this is what happens:

```
You type a command  
↓  
Shell reads the command  
↓  
Computer runs the command  
↓  
Terminal shows the result
```

Example command:

```
echo "I am learning terminal"
```

Output:

```
I am learning terminal
```

In this example:

```
echo = command  
"I am learning terminal" = text given to the command
```

Simple meaning:

A command is an instruction you type for the computer.

11. What Is `echo` ?

`echo` is a simple command that prints text to the terminal.

Example:

```
echo "Hello"
```

Output:

```
Hello
```

Another example:

```
echo "I am learning Sprint 2"
```

Output:

```
I am learning Sprint 2
```

Simple meaning:

`echo` displays text in the terminal.

This is a safe beginner command because it does not delete or change files.

12. Command History

The terminal remembers commands you typed before.

This is called **command history**.

Command history is useful because you do not have to type the same command again and again.

You can move through command history using:

Up arrow
Down arrow

13. Up Arrow

The **Up arrow** shows the previous command you typed.

Example:

First you type:

```
echo "Hello"
```

Then you press:

```
Up arrow
```

The terminal shows the previous command again:

```
echo "Hello"
```

Simple meaning:

Up arrow helps you bring back older commands.

Why it is useful:

- Saves time
 - Avoids retyping
 - Helps correct mistakes
 - Helps repeat commands
-

14. Down Arrow

The **Down arrow** moves forward through command history.

If you pressed Up arrow many times, Down arrow helps you move back toward newer commands.

Simple meaning:

Down arrow moves forward in your command history.

Together:

```
Up arrow = older command  
Down arrow = newer command
```

15. Tab Auto-Complete

`Tab` helps auto-complete commands, file names, or folder names.

Example:

Suppose you have a folder called:

```
my-project
```

You type:

```
cd my
```

Then press:

```
Tab
```

The terminal may complete it as:

```
cd my-project
```

Simple meaning:

Tab helps you type faster and avoid spelling mistakes.

Why Tab is useful:

- You do not need to type long names fully
- It reduces spelling mistakes

- It helps you find existing files and folders
- It makes terminal work faster

Important note:

Tab behavior can be different depending on your shell and terminal.

Sometimes pressing Tab once completes the name.

Sometimes pressing Tab twice shows suggestions.

16. Clearing the Terminal Screen

Sometimes the terminal becomes messy because there is too much text on the screen.

You can clear the view.

Clearing the terminal screen does **not** delete your files.

It only cleans the terminal display.

In Bash, Git Bash, macOS, or Linux

Use:

```
Ctrl + L
```

In Windows PowerShell or Command Prompt

Use:

```
cls
```

Simple meaning:

Clearing the screen only makes the terminal view clean. It does not remove your files or commands from the computer.

Comparison:

```
Bash / Git Bash / macOS / Linux = Ctrl + L  
PowerShell / Command Prompt = cls
```

17. Ctrl + C: Stop a Running Command

Sometimes a command keeps running.

To stop a running command, press:

```
Ctrl + C
```

Simple meaning:

Ctrl + C interrupts or stops a running command.

Example practice command on Windows PowerShell:

```
ping 127.0.0.1 -t
```

Then press:

```
Ctrl + C
```

Example practice command on macOS, Linux, or Git Bash:

```
ping 127.0.0.1
```

Then press:

```
Ctrl + C
```

What does `ping` do?

`ping` checks network communication with an address.

For practice, `127.0.0.1` means your own computer.

You do not need to deeply understand networking now. Here, `ping` is only used as a safe example of a command that keeps running until you stop it.

Important note:

In PowerShell, `Ctrl + C` may also copy selected text if text is selected.

18. Ctrl + Z: Pause or Background a Process

In many Unix-like terminals, such as Bash, this shortcut can pause a running process or send it to the background:

```
Ctrl + Z
```

Simple meaning:

Ctrl + Z can pause or background a running process in many Unix-like shells.

This is more advanced than `Ctrl + C`.

For beginners:

```
Ctrl + C = stop the command  
Ctrl + Z = pause/background the command
```

Important note:

`Ctrl + Z` may behave differently in PowerShell or Command Prompt.

Do not worry if it does not work the same way on your system.

19. fg: Bring a Background Process Back

If a command was paused or sent to the background with `Ctrl + Z`, you can bring it back to the front using:

```
fg
```

`fg` means **foreground**.

Simple meaning:

`fg` brings a paused/background command back to the front.

Example idea:

```
Ctrl + Z = send/pause process  
fg = bring it back
```

Beginner note:

This works mainly in Unix-like shells such as Bash.

20. `!!`: Repeat the Last Command

In many Bash-like shells, this repeats the previous command:

```
!!
```

Example:

```
echo "Hello"  
!!
```

The second command repeats:

```
echo "Hello"
```

Simple meaning:

`!!` means run the last command again.

Why it is useful:

- Saves time
- Quickly repeats a command
- Helpful when you forgot to add something before a command

Important note:

!! may not work in every terminal or shell.

It is common in Bash-like shells.

21. Windows, macOS, and Linux Terminal Differences

Different operating systems use different terminal tools and shells.

Windows

Windows users may use:

```
PowerShell  
Command Prompt  
Windows Terminal  
Git Bash
```

PowerShell commands can sometimes be different from Bash commands.

Command Prompt is older and simpler.

Windows Terminal is a modern terminal application that can open different shells.

Git Bash gives Windows users a Bash-like environment.

macOS

macOS users commonly use:

```
Terminal
```

Some users install:

```
iTerm
```

macOS often uses shells like Zsh or Bash depending on setup.

Linux

Linux users may use many terminal applications depending on the distribution and desktop environment.

Examples:

```
Terminal  
Kitty  
Ghostty
```

Linux commonly uses Bash or other Unix-like shells.

Simple meaning:

The terminal app may change depending on the operating system, but the basic idea is the same: type commands and read output.

22. Opening the Terminal in VS Code

Since this course uses VS Code, you can open a terminal inside VS Code.

Steps:

1. Open VS Code.
2. Open your project folder.
3. Go to the top menu.
4. Click:

```
Terminal → New Terminal
```

A terminal will appear at the bottom of VS Code.

Simple meaning:

VS Code has a built-in terminal, so you can type commands without leaving VS Code.

This is useful because you can:

- Write code in VS Code
 - Use terminal commands in the same window
 - Manage project files
 - Run Git commands later
 - Run database commands later
-

23. Safe Beginner Commands in Sprint 2

In Sprint 2, use only safe commands.

Safe command example:

```
echo "I am learning Sprint 2"
```

This only prints text.

Another safe action:

```
Ctrl + L
```

This only clears the terminal screen.

Another safe command on PowerShell:

```
cls
```

This only clears the screen.

Commands to be careful with later:

```
rm  
sudo  
delete  
format
```

Beginner warning:

Do not run commands you do not understand, especially commands that delete files, change permissions, or run with admin power.

You will learn file commands carefully in Sprint 3.

24. Important Sprint 2 Commands and Shortcuts Summary

Command / Shortcut	Meaning	Where it is commonly used
Up arrow	Show previous command	Most terminals
Down arrow	Move forward in command history	Most terminals
Tab	Auto-complete command/path/file/folder	Most terminals
Ctrl + L	Clear terminal screen	Bash, macOS, Linux, Git Bash
cls	Clear terminal screen	PowerShell, Command Prompt
Ctrl + C	Stop/interrupt a running command	Most terminals
Ctrl + Z	Pause/background process	Many Unix-like shells
fg	Bring background process to foreground	Bash/Unix-like shells
!!	Repeat last command	Many Bash-like shells
echo "Hello"	Print text	Many shells

25. Sprint 2 Practice Task

Complete this practice task step by step.

Step 1: Open VS Code

Open VS Code on your computer.

Step 2: Open your project folder

Open the folder you created in Sprint 1.

Example:

```
my-project
```

Step 3: Open the VS Code terminal

Click:

```
Terminal → New Terminal
```

Step 4: Type a simple command

Type:

```
echo "I am learning Sprint 2"
```

Press Enter.

You should see:

```
I am learning Sprint 2
```

Step 5: Use Up arrow

Press:

```
Up arrow
```

You should see the previous command again.

Step 6: Use Down arrow

Press:

```
Down arrow
```

This moves forward in your command history.

Step 7: Try Tab auto-complete

Start typing a folder or file name and press:

```
Tab
```

Watch if the terminal completes the name or shows suggestions.

Step 8: Clear the screen

For Bash, Git Bash, macOS, or Linux:

```
Ctrl + L
```

For PowerShell or Command Prompt:

```
cls
```

Step 9: Stop a running command

On Windows PowerShell, run:

```
ping 127.0.0.1 -t
```

Then press:

```
Ctrl + C
```

On macOS, Linux, or Git Bash, run:

```
ping 127.0.0.1
```

Then press:

```
Ctrl + C
```

26. Sprint 2 Checkpoint

The checkpoint question is:

What is the difference between terminal, command line, and shell?

A good beginner answer:

```
A terminal is the app I open.  
The command line is where I type commands.  
The shell is the program that understands and runs those commands.
```

You are ready to move forward only when you can explain this without memorizing complex words.

27. Common Beginner Mistakes

Mistake 1: Thinking terminal and shell are the same

They are related, but they are not exactly the same.

Correct idea:

```
Terminal = app/window  
Shell = program inside that understands commands
```

Mistake 2: Thinking the command line is a different app

The command line is not usually a separate app.

It is the place inside the terminal where you type commands.

Correct idea:

```
Terminal = app  
Command line = typing area inside the terminal
```

Mistake 3: Being afraid of the terminal

The terminal looks technical, but you can start safely.

In Sprint 2, you are mostly using safe actions like:

```
echo "Hello"
```

and:

```
Ctrl + L
```

Mistake 4: Running unknown commands from the internet

Do not copy and run terminal commands if you do not understand what they do.

Be careful with commands like:

```
rm
sudo
format
delete
```

These can be dangerous if used incorrectly.

Mistake 5: Forgetting that Windows and Bash commands can differ

Example:

```
Bash clear screen = Ctrl + L
PowerShell clear screen = cls
```

So always notice which terminal or shell you are using.

Mistake 6: Thinking clearing the screen deletes files

Clearing the terminal screen does not delete files.

It only cleans the terminal view.

Mistake 7: Confusing Ctrl + C and Ctrl + Z

Simple difference:

```
Ctrl + C = stop/interrupt a command
Ctrl + Z = pause/background a command in many Unix-like shells
```

28. Key Terms Summary

Term	Simple Meaning
Terminal	The app/window used to type commands
Terminal emulator	A modern app that behaves like a terminal
Command line	The text area where commands are typed
Shell	The program that understands and runs commands
Bash	A common Unix-like shell

Term	Simple Meaning
Command	An instruction typed for the computer
Command history	Previous commands remembered by the terminal/shell
Auto-complete	A feature that completes commands or file/folder names
Ctrl + C	Stops a running command
Ctrl + Z	Pauses/backgrounds a command in many Unix-like shells
fg	Brings a background process back to the front
!!	Repeats the previous command in many Bash-like shells
cls	Clears the screen in PowerShell/Command Prompt
Ctrl + L	Clears the screen in many Unix-like terminals
echo	Prints text to the terminal

29. Sprint 2 Final Understanding

After Sprint 2, you should understand that the terminal is not something to fear.

It is simply a tool for typing commands.

The command line is where you type.

The shell is what understands what you typed.

The terminal shows the result.

You should now understand this flow:

```
You type a command
↓
Shell understands it
↓
Computer runs it
↓
Terminal shows output
```

You should also be able to use basic terminal keyboard actions:


```
Up arrow = previous command
Down arrow = newer command
Tab = auto-complete
Ctrl + L = clear screen in Bash-like terminals
cls = clear screen in PowerShell/Command Prompt
Ctrl + C = stop command
Ctrl + Z = pause/background command in many Unix-like shells
fg = bring background command to front
!! = repeat last command in many Bash-like shells
```

30. Sprint 2 Mini Assessment

Answer these questions before moving to Sprint 3.

1. What is a command line?
2. What is a terminal?
3. What is a terminal emulator?
4. What is a shell?
5. What is Bash?
6. What does the Up arrow do?
7. What does the Down arrow do?
8. What does Tab do?
9. How do you clear the terminal in Bash/Git Bash/macOS/Linux?
10. How do you clear the terminal in PowerShell or Command Prompt?
11. How do you stop a running command?
12. What does Ctrl + Z do in many Unix-like shells?
13. What does `fg` do?
14. What does `!!` do?
15. What does `echo` do?
16. Explain terminal vs shell in one sentence.
17. Explain terminal vs command line in one sentence.
18. Why should beginners avoid running unknown commands from the internet?

31. Sprint 2 Completion Checklist

Tick these when completed:

- ```
[] I opened a terminal.
[] I opened the terminal inside VS Code.
[] I typed echo "I am learning Sprint 2".
[] I used the Up arrow to show a previous command.
[] I used the Down arrow to move forward in command history.
```

```
[] I tried Tab auto-complete.
[] I cleared the terminal screen.
[] I stopped a running command using Ctrl + C.
[] I understand what Ctrl + Z does in many Unix-like shells.
[] I understand what fg does.
[] I understand what !! does.
[] I can explain what a command line is.
[] I can explain what a terminal is.
[] I can explain what a terminal emulator is.
[] I can explain what a shell is.
[] I can explain what Bash is.
[] I can explain terminal vs shell in simple words.
```

---

## 32. Final Sprint 2 Checkpoint Answer Example

Question:

What is the difference between terminal, command line, and shell?

Good answer:

A terminal is the app or window I open to type commands.  
The command line is the place where I type those commands.  
The shell is the program that understands and runs the commands.

Short answer:

Terminal = window  
Command line = typing place  
Shell = command interpreter

---

## End of Sprint 2 Notes

You are ready to move to **Sprint 3: Bash File and Folder Commands** only after you can:

1. Explain terminal, command line, and shell clearly.
2. Use command history with Up and Down arrows.
3. Use Tab auto-complete.
4. Clear the screen.

5. Stop a running command with Ctrl + C.
6. Understand the basic idea of Ctrl + Z, `fg`, and `!!`.

Sprint 3 will teach you how to navigate folders and manage files using Bash commands.